

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем

Лабораторная работа №6

по дисциплине: Основы программирования

тема: **«Введение в функции»**

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич

Проверили:
ст. преп. Притчин Иван Сергеевич
асс. Черников Сергей Викторович
Рядинский Данил

Белгород 2022 г.

Цель работы: получение навыков написания функций при решении простых задач. Закрепление навыков разработки алгоритмов разветвляющейся и циклической структуры. Получение навыков формулирования спецификаций к разрабатываемым функциям.

Содержание отчета:

Тема лабораторной работы.

Цель лабораторной работы.

Решения задач. Для каждой задачи указать:

- Условие задачи.
- Тестовые данные
- Исходный код функции и её спецификацию.

Задачи с двумя звездочками допускается не решать.

Задание №1 Напишите функцию *abs* для вычисления модуля вещественного числа *x*.

1. Заголовок: `float abs(float a);`

2. Назначение: возвращает модуль вещественного числа *a*.

Код программы:

```
//возвращает модуль вещественного числа a.
#include <stdio.h>

double abs(double a) {
    return a >= 0 ? a : -a;
}

int main() {
    double a;
    scanf("%lf", &a);

    printf("%lf", abs(a));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
1.23	1.23
-10000.23	10000.23

Задание №2 Напишите функцию sign.

1. Заголовок: `int sign(int a);`
2. Назначение: определяет положительное, отрицательное или 0 число a.

Код программы:

```
//определяет положительное, отрицательное или 0 число a.
#include <stdio.h>

int sign(int a) {
    if (a > 0) {
        return 1;
    } else if (a == 0) {
        return 0;
    } else {
        return -1;
    }
}

int main() {
    int x;
    scanf("%d", &x);

    printf("%d", sign(x));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
0	0
12	1
-12	-1

Задание №3 Напишите функцию *max2*, которая возвращает максимальное значение из двух целочисленных переменных типа *int*.

1. Заголовок: `long long max2(long long a, long long b).`

2. Назначение:находит максимум из целочисленных чисел а и b.

Код программы:

```
//находит максимум из целочисленных чисел а и b.
#include <stdio.h>

int max2(int a, int b) {
    return a > b ? a : b;
}

int main() {
    int a, b;
    scanf("%d %d", &a, &b);

    printf("Maximum:%d", max2(a, b));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
1 5	Maximum:5
50 16	Maximum:50

Задание №4 Напишите функцию *max3*, которая возвращает максимальное значение из трёх целочисленных переменных типа *int*.

1. Заголовок: `long long max3(long long a, long long b, long long c)`

2. Назначение:находит максимум из целочисленных чисел а, b и c.

Код программы:

```
//находит максимум из целочисленных чисел а, b и c.
#include <stdio.h>

int max2(int a, int b) {
    return a > b ? a : b;
}

int max3(int a, int b, int c) {
    return max2(max2(a, b), c);
}

int main() {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    printf("Maximum:%d", max3(a, b, c));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
1 2 3	Maximum:3
12 34 2	Maximum:34
56 17 8	Maximum:56

Задание №5 Напишите функцию *getDistance*, которая вычисляет расстояние между двумя точками, заданными целочисленными координатами (x_1, y_1) , (x_2, y_2) .

1. Заголовок: `long long getDistance(long long x1, long long y1, long long x2, long long y2)`

2. Назначение: вычисляет расстояние между точками, заданными целочисленными координатами (x_1, y_1) , (x_2, y_2) .

Код программы:

```
//вычисляет расстояние между точками, заданными целочисленными координатами
(x1, y1), (x2, y2).
#include <stdio.h>
#include <math.h>
#include <windows.h>

double getDistance(double x1, double y1, double x2, double y2) {
    return (sqrt(pow((x2 - x1), 2) + pow((y2 - y1), 2)));
}

int main() {
    SetConsoleOutputCP(CP_UTF8);

    printf("Ввод x1, y1, x2, y2:");
    double x1, y1, x2, y2;
    scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2);

    printf("distance:%lf", getDistance(x1, y1, x2, y2));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
Ввод x1, y1, x2, y2: 2 3 1 4	distance:1.414214
Ввод x1, y1, x2, y2: 1 1 1 1	distance:0
Ввод x1, y1, x2, y2: 1 3 5 6	distance:5.000000
Ввод x1, y1, x2, y2: 60 60 50 50	distance: 14.142136

Задание №6 Напишите функцию *solveX2*, которая выводит корни квадратного уравнения: $ax^2 + bx + c = 0$ ($a \neq 0$)

1. Заголовок: `long long solveX2(long long a, long long b, long long c)`

2. Назначение: выводит корни квадратного уравнения, после ввода a, b и c.

Код программы:

```
//выводит корни квадратного уравнения, после ввода a, b и c.
#include <stdio.h>
#include <math.h>

void solveX2(double a, double b, double c) {
    double D = b * b - 4 * a * c;
    if (D > 0) {
        double sqrtD = sqrt(D);
        double x1 = (-b - sqrtD) / (2 * a);
        double x2 = (-b + sqrtD) / (2 * a);

        printf("%lf %lf", x1, x2);
    } else if (D == 0) {
        double x = -b / 2 * a;

        printf("%lf", x);
    } else {
        printf("D < 0");
    }
}

int main() {
    double a, b, c;
    scanf("%lf %lf %lf", &a, &b, &c);

    solveX2(a, b, c);

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
1 2 1	-1.000000
12 3 2	D < 0
5 12 3	-2.116515 -0.283485

Задание №7 Написать функцию *isDigit*, которая возвращает значение 'истина', если символ *x* является цифрой, 'ложь' - в противном случае.

1. Заголовок: `long long isDigit(long long a)`

2. Назначение: возвращает значение "истина", если символ *a* является цифрой, иначе "ложь".

Код программы:

```
//возвращает значение "истина", если символ a является цифрой, иначе "ложь".
#include <stdio.h>

int isDigit(int a) {
    return a >= 0 && a < 10 ? 1 : 0;
}

int main() {
    int x;
    scanf("%d", &x);

    int check = isDigit(x);

    if (check == 1) {
        printf("True");
    } else {
        printf("False");
    }

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
12	False
5	True
A	False

Задание №8 Напишите функцию *swap*, которая принимает две переменные типа *float* и обменивает их значения

1. Заголовок: `void swap(float *a, float *b)`

2. Назначение: обменивает значения переменных *a* и *b* типа *float*.

Код программы:

```
//обменивает значения переменных a и b типа float.
#include <stdio.h>

void swap(float *a, float *b) {
    float t = *a;
    *a = *b;
    *b = t;
}

int main() {
    float a, b;
    scanf("%f %f", &a, &b);

    swap(&a, &b);

    printf("%f %f", a, b);

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
15.5 4.5	4.500000 15.500000
1.999999 22.44421	22.444210 1.999999
2 14	14.000000 2.000000

Задание №9 Напишите функцию *sort2*, которая упорядочивает значения *a* и *b* типа *float*. Т.е. если $a > b$ то после выполнения функции значение переменной *a* должно быть не больше значения переменной *b*.

1. Заголовок: `void swap(float *a, float *b)`

2. Назначение: обменивает значения переменных *a* и *b* типа *float*.

Код программы:

```
//обменивает значения переменных a и b типа float.
#include <stdio.h>

void swap(float *a, float *b) {
    float t = *a;
    *a = *b;
    *b = t;
}

void sort2(float *a, float *b) {
    if (*a > *b) {
        swap(a, b);
    }
}

int main() {
    float a, b;
    scanf("%f %f", &a, &b);

    sort2(&a, &b);

    printf("%f %f", a, b);

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
33.16 4.5	4.500000 33.160000
5.6 5.4	5.400000 5.600000
17 18	17.000000 18.000000

Задание №10 Напишите функцию *sort3*, которая упорядочивает значения переменных *a, b, c* типа *float* таким образом, чтобы: $a \leq b \leq c$

1. Заголовок: `void sort3(float *a, float *b, float *c)`

2. Назначение: обменивает значения переменных a, b и c типа float.

Код программы:

```
//обменивает значения переменных a, b и c типа float.
#include <stdio.h>

void swap(float *a, float *b) {
    float t = *a;
    *a = *b;
    *b = t;
}

void sort2(float *a, float *b) {
    if (*a > *b) {
        swap(a, b);
    }
}

void sort3(float *a, float *b, float *c) {
    sort2(a, b);
    sort2(b, c);
    sort2(a, b);
}

int main() {
    float a, b, c;
    scanf("%f %f %f", &a, &b, &c);

    sort3(&a, &b, &c);

    printf("%f %f %f", a, b, c);

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
2 3 4	2.000000 3.000000 4.000000
3.4 5.7 1.2	1.200000 3.400000 5.700000
12.4 12.3 12	12.000000 12.300000 12.400000

Задание №11 Написать функцию, которая возвращает значение 'истина', если можно составить треугольник с целочисленными сторонами a, b, c ($a, b, c \in N$), 'ложь' - в противном случае.

1. Заголовок: `bool isTrianglePossible(int a, int b, int c)`

2. Назначение: возвращает "истина" если из целочисленных a, b и c можно составить треугольник, иначе "ложь".

Код программы:

```
//возвращает "истина" если из целочисленных a, b и c можно составить треугольник,
//иначе "ложь".
#include <stdio.h>
#include <stdbool.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void sort2(int *a, int *b) {
    if (*a > *b) {
        swap(a, b);
    }
}

void sort3(int *a, int *b, int *c) {
    sort2(a, b);
    sort2(b, c);
    sort2(a, b);
}

bool isTrianglePossible(int a, int b, int c) {
    sort3(&a, &b, &c);

    if (a + b > c) {
        return 1;
    } else {
        return 0;
    }
}

int main() {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    if (isTrianglePossible(a, b, c)) {
        printf("True");
    } else {
        printf("False");
    }

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
1 2 3	False
2 3 4	True
15 14 10	True

Задание №12 Напишите функцию *getTriangleTypeLength*, которая возвращает значение 0, если треугольник со сторонами *a*, *b*, *c* является остроугольным, 1 – если прямоугольным, 2 – тупоугольным, -1 – если треугольник с такими сторонами не существует.

1. Заголовок: `getTriangleTypeLength(int a, int b, int c)`

2. Назначение: определяет тип треугольника по его сторонам *a*, *b* и *c*

Код программы:

```
//определяет тип треугольника по его сторонам a, b и c
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void sort2(int *a, int *b) {
    if (*a > *b) {
        swap(a, b);
    }
}

void sort3(int *a, int *b, int *c) {
    sort2(a, b);
    sort2(b, c);
    sort2(a, b);
}

int getTriangleTypeLength(int a, int b, int c) {
    sort3(&a, &b, &c);

    if (a + b <= c) {
        return -1;
    } else if (a * a + b * b > c * c) {
        return 0;
    } else if (a * a + b * b < c * c) {
        return 2;
    } else {
        return 1;
    }
}

int main() {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    printf("%d", getTriangleTypeLength(a, b, c));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
3 4 5	1
14 15 20	0
13 14 25	2
1 2 3	-1

Задание №13 Напишите функцию *isPrime*, которая возвращает значение 'истина', если число является простым, иначе – 'ложь'. Приложите 3 вариации:

(a) Без оптимизаций

(b) С оптимизацией перебора до $\sqrt{N}$.

(c) С оптимизацией перебора до $\sqrt{N}$ и шагом 2.

1. Заголовок: `bool isPrime(int a)`

2. Назначение: определяет является ли число a простым

Код программы:

(a)

```
#include <stdio.h>
#include <stdbool.h>

bool isPrime(int a) {
    int flag = 1;
    for (int i = 2; i < a; i++) {
        if (a % i == 0) {
            flag = 0;
            break;
        }
    }
    return flag == 1 ? 1 : 0;
}

int main() {
    int a;
    scanf("%d", &a);

    if (isPrime(a)) {
        printf("True");
    } else {
        printf("False");
    }
}
```

(b)

```
#include <stdio.h>
#include <math.h>

int isPrime(int a) {
    int d = 3;
    int maxNumber = sqrt(a);
    int check = !(a == 1 || a % 2 == 0 && a != 2);
    while (d <= maxNumber && check) {
        check = a % d;
    }

    if (check) {
        return 1;
    } else {
        return 0;
    }
}

int main() {
    int a;
    scanf("%d", &a);

    if (isPrime(a)) {
        printf("True");
    } else {
        printf("False");
    }
}
```

(c)

```
#include <stdio.h>
#include <math.h>

int isPrime(int a) {
    int d = 3;
    int maxNumber = sqrt(a);
    int check = !(a == 1 || a % 2 == 0 && a != 2);
    while (d <= maxNumber && check) {
        check = a % d;
        d += 2;
    }

    if (check) {
        return 1;
    } else {
        return 0;
    }
}

int main() {
    int a;
    scanf("%d", &a);

    if (isPrime(a)) {
        printf("True");
    } else {
        printf("False");
    }
}
```

Тестирование:

Входные данные	Выходные данные
3	True
12	False
17	False

Задание №15 Найти количество чисел-палиндромов от 1 до n.

1. Заголовок: `int checkingForPalindromeNumber(int a)`

2. Назначение: определяет является ли число a палиндромом

1. Заголовок: `numberOfPalindromes(int n)`

2. Назначение: определяется сколько чисел палиндромов в промежутке от 1 до n

Код программы:

```
#include <stdio.h>

//определяет является ли число a палиндромом
int checkingForPalindromeNumber(int a) {
    int lastNumber = a;

    int newNumber = 0;
    while (a > 0) {
        newNumber = newNumber * 10 + a % 10;
        a = a / 10;
    }

    if (lastNumber == newNumber) {
        return 1;
    } else {
        return 0;
    }
}

//определяется сколько чисел палиндромов в промежутке от 1 до n
int numberOfPalindromes(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        if (checkingForPalindromeNumber(i)) {
            count++;
        }
    }

    return count;
}

int main() {
    int n;
    scanf("%d", &n);

    printf("%d", numberOfPalindromes(n));

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
3	0
11	1
121	12

Задание №16 В шестизначных автобусных билетах найти счастливые.

1. Заголовок: `int checkingForThePerfectNumber(int a)`

2. Назначение: определяет, является ли билет а счастливым(билет называется счастливым, если сумма первых трёх цифр равняется сумме последних трёх цифр)

Код программы:

```
#include <stdio.h>

//определяет, является ли билет а счастливым
int sumOfTheLastThree = 0;
for (int i = 1; i <= 3; i++) {
    sumOfTheLastThree += a % 10;
    a /= 10;
}

int sumOfTheFirstThree = 0;
for (int i = 1; i <= 3; i++) {
    sumOfTheFirstThree += a % 10;
    a /= 10;
}

if (sumOfTheFirstThree == sumOfTheLastThree) {
    return 1;
} else {
    return 0;
}
}

int main() {
    int n;
    scanf("%d", &n);

    if (checkingForThePerfectNumber(n)) {
        printf("Good");
    } else {
        printf("Bad");
    }

    return 0;
}
```

Тестирование:

Входные данные	Выходные данные
121121	Good
123600	Good
333444	Bad

Ревью:

4. условие в новой методичке другое

6. т.к. функция ничего не возвращает, пишешь `void`, так же хз насчет `d=0`, у тебя в выводе будет два одинаковых корня и это по идее неправильно

11. а зачем тебе условие $(a * a + b * b == c * c)$?

12. сделай проверку на существование треугольника первой, так не будет дублирования кода в остальных условиях. Забыл, по условию функция возвращает, поэтому `return` и в заголовке `int` вместо `bool`

15. вроде все ок

16. ложно решена, ты же знаешь, что числа шестизначные, ну запусти ты два одинаковых цикла на три повторения и найди суммы первых и последних. Еще в тестах желательно написать случай , когда начало и конец не зеркальны, но сумма у них одинакова